# WHNet · arquitectura del modelo

EfficientADMMUnrolledNet · ADMM aprendido con física dipolar

## A FLUJO GENERAL

**Entradas**

$\varphi,\ M,\ D,\ W$

$W = M$ si se omite

**Normalización**

$s = \max(\mathrm{RMS}_M(\varphi), 10^{-3})$

$p = \varphi / s$

**Inicialización**

$\chi^0 = h^0 = 0$

$\eta_y^0 = \eta_u^0 = \eta_v^0 = 0$

**K iteraciones ADMM**

$\mathcal{B}_0 \longrightarrow \cdots \longrightarrow \mathcal{B}_{K-1}$

Dos U-Nets 3D por iteración

**Restaurar escala**

$\widehat{\chi} = s\chi^K$

$\widehat{\varphi}_h = sh^K$

**Salidas**

Susceptibilidad $\widehat{\chi}$

Campo de fondo $\widehat{\varphi}_h$

$p, M, D, W$ disponibles en cada iteración · K configurable · volúmenes [N, C, D, H, W]

## B UNA ITERACIÓN $\mathcal{B}_k$

Flechas: orden de cálculo; las ecuaciones indican las dependencias.

Estado entrante: $\chi^k, h^k, \eta_y^k, \eta_u^k, \eta_v^k$ y $\delta_k$ · $h \equiv \varphi_h$ en escala normalizada

**1 · Física y residuo**

$f^k = A(\chi^k)$

$r_k = \mathrm{RMS}_M(f^k + h^k - p)$

**2 · Consistencia con los datos**

$$y = \frac{Wp + f^k + h^k + \eta_y^k}{W + 1}$$

**3 · Redes proximales aprendidas**

$u = P_\chi(\chi^k + \eta_u^k;\ k, \delta_k)$

$v = P_h(h^k + \eta_v^k;\ k, r_k)$

chinet / phinet · dos U-Nets 3D

**4 · Actualizar susceptibilidad**

$$q = \frac{D\,\mathcal{F}(y - \eta_y^k - h^k) + \mathcal{F}(u - \eta_u^k)}{D^2 + 1 + \varepsilon}$$

$\chi^{k+1} = M\,\mathcal{F}^{-1}(q)$

**5 · Recalcular campo dipolar**

$f^{k+1} = A(\chi^{k+1})$

Se usa χ después de la máscara

Física en FP32 · FFT real

**6 · Actualizar campo de fondo**

$h^{k+1} = \frac{1}{2}(y - \eta_y^k - f^{k+1} + v - \eta_v^k)$

**7 · Actualizar variables duales**

$\eta_y^{k+1} = \eta_y^k + f^{k+1} + h^{k+1} - y$

$\eta_u^{k+1} = \eta_u^k + \chi^{k+1} - u$

$\eta_v^{k+1} = \eta_v^k + h^{k+1} - v$

**Estado para la siguiente iteración**

$\chi^{k+1}, h^{k+1}, \eta_y^{k+1}, \eta_u^{k+1}, \eta_v^{k+1}$

$\delta_{k+1} = \mathrm{RMS}_M(\chi^{k+1} - \chi^k)$

Inicio: $\delta_0 = 0$

Redes: $P_\chi$ y $P_h$ tienen parámetros independientes; cada una comparte sus pesos entre las K iteraciones.

Cada proximal aplica una corrección residual $P(x; k, a) = x + \alpha(k, a)\,U(x)$; una MLP controla la ganancia $\alpha$.

$A(x) = \mathcal{F}^{-1}[D\,\mathcal{F}(x)]$ · $\mathrm{RMS}_M(x) = \sqrt{\frac{\sum Mx^2}{\max(\sum M,\,1)} + \varepsilon^2}$ · $\mathcal{F}$: FFT real 3D

Física dipolar · Redes aprendidas · Actualizaciones algebraicas · Datos / estado

Fuente: src/whnet/model.py · configuración actual de entrenamiento · normalización activada